Proyecto01: Informe Técnico Ejecutivo - Etapa 01

Binjie Liang
Redes - Tecnológico Costa Rica
bliang@estudiantec.cr

William Jiménez Venegas
Redes - Tecnológico Costa Rica
wijimenez@estudiantec.cr

Jose David Serrano Robles
Redes - Tecnológico Costa Rica
josserrano@estudiantec.cr

Mariana Navarro Jiménez
Redes - Tecnológico Costa Rica
mariana12@estudiantec.cr

Esteban Daniel Secaida Vargas
Redes - Tecnológico Costa Rica
esecaida@estudiantec.cr

Miguel Alejandro Madrigal Ramírez
Redes - Tecnológico Costa Rica
alemarra99@estudiantec.cr

***Abstract*—El presente informe propone el diseño modular de un sistema de transferencia de archivos TXT, PDF y PNG entre dos terminales basadas en Arduino Uno y Arduino Nano, interconectadas mediante un enlace de cobre blindado (STP) con comunicación Full Duplex. La arquitectura se organiza en las capas de aplicación, red, enlace de datos y física del modelo OSI. Se propone el protocolo Go-Back-N con ventanas configurables, detección de errores mediante Checksum de 16 bits y corrección de errores de un bit por bloque mediante Hamming extendido (13,8). Para atender las limitaciones de memoria de los microcontroladores, la propuesta contempla la gestión de ventanas en las computadoras y el procesamiento incremental de datos en los Arduino. Se describen los módulos, la estructura de las tramas y los procedimientos de establecimiento, transferencia y finalización, como base para su posterior implementación y validación.**

***Index Terms*—Arduino, capa física, capa de enlace de datos, detección de errores, corrección de errores, redes, transferencia de archivos, ventana deslizante**

I. Introducción

La transferencia de archivos entre dispositivos representa una de las aplicaciones fundamentales de las redes de computadoras. Para poder realizar este proceso, es necesario establecer mecanismos de comunicación que permitan transmitir la información de manera organizada y confiable, incluso ante alteraciones de los medios físicos. Con esto en mente, los protocolos de comunicación establecen las reglas que se deben seguir para que este intercambio de datos funcione de manera correcta entre los dispositivos. Establecen cómo se envían, reciben e interpretan los datos y qué procedimientos seguir cuando surge un problema de comunicación. Por ello se incorporan mecanismos de detección de errores y control de flujo que contribuyen a una transferencia confiable. [1]

El presente proyecto plantea el diseño de un sistema de transferencia de archivos entre dos terminales, utilizando un Arduino Uno y un Arduino Nano, interconectados mediante un enlace de cobre blindado (STP) con capacidad Full Duplex de comunicación. El sistema deberá permitir la transferencia de archivos de texto plano, documentos PDF e imágenes PNG, utilizando una arquitectura modular basada en el modelo OSI. Esta organización facilita la delimitación de responsabilidades de cada componente, la creación de interfaces entre módulos, la implementación y validación de la solución.

Uno de los principales desafíos del proyecto consiste en garantizar la integridad de la información durante su transmisión a través de un medio que es susceptible a interferencias y a errores. Para ello, se propone utilizar Hamming extendido (13,8) en la capa física, capaz de corregir un error de bit y detectar errores dobles por bloque, y un Checksum de 16 bits en la capa de enlace de datos para detectar alteraciones en las tramas. El protocolo de ventana deslizante Go-Back-N gestionará el flujo de información, las confirmaciones acumulativas y las retransmisiones. Estos mecanismos buscarán proporcionar una comunicación confiable y aprovechar la capacidad bidireccional del enlace. [1], [2]

En esta primera etapa el objetivo principal es desarrollar el diseño técnico de la solución, definiendo la arquitectura del sistema, los protocolos correspondientes a las capas y sus funciones. Para ello se elaborarán diagramas de bloques en tres niveles de descomposición, diagramas de secuencia del protocolo de comunicación y la especificación detallada del formato de las tramas. Este diseño servirá como base para la siguiente etapa, en la que se implementará el sistema y se evaluará su comportamiento bajo diferentes velocidades de transmisión, tamaños de carga y anchos de ventana.

II. Diseño Propuesto

A. Estructuración del diseño basado en diseño modular

El diseño del sistema se fundamenta en la metodología de diseño modular (Fig. 7), mediante la cual se divide una solución compleja en componentes con funciones y responsabilidades específicas. Esto permite analizar cada módulo de manera independiente, establecer interfaces de comunicación entre ellos y facilitar su posterior implementación y validación. Para el presente proyecto, se utiliza el modelo de referencia OSI como base conceptual para organizar los protocolos y las funciones necesarias para transferir archivos entre las dos terminales. [1]

1) Arquitectura general del sistema: La solución contempla dos terminales, cada una integrada por una computadora y un microcontrolador Arduino. La primera terminal utilizará un Arduino Uno y la segunda un Arduino Nano. Ambos dispositivos estarán interconectados mediante un enlace de cobre blindado STP, que permitirá transmitir y recibir información simultáneamente gracias a su capacidad Full Duplex. Ver Fig. 8

Cada terminal podrá actuar como emisora o receptora, de acuerdo con la operación solicitada por el usuario. La computadora proporcionará la interfaz gráfica para seleccionar archivos, configurar los parámetros de comunicación y visualizar el estado de la transferencia. Por su parte, los microcontroladores participarán en la transmisión y recepción de información a través del enlace físico. Ver Fig. 9

2) Organización por capas: La arquitectura propuesta contempla cuatro capas funcionales del modelo OSI: aplicación, red, enlace de datos y física. Esta organización permite establecer una separación entre la gestión de archivos, el manejo de paquetes, el control de la transmisión y la comunicación mediante el medio físico. Aunque el sistema establece una conexión punto a punto entre dos dispositivos y no requiere mecanismos de enrutamiento, se contempla la capa de red como parte de la arquitectura modular.

La **capa de aplicación** será responsable de gestionar los archivos que se desean transferir y proporcionar los mecanismos de interacción con el usuario. A su vez, esto permite que exista una configuración de los parámetros de transmisión y visualizar la información sobre el estado del enlace, el porcentaje de transferencia y la tasa estimada de errores.

La **capa de red** tendrá como responsabilidad gestionar los paquetes de información que se intercambian entre las terminales y proporcionar una interfaz entre la capa de aplicación y la capa de enlace de datos. Debido a que únicamente existen dos dispositivos conectados directamente, no será necesario implementar algoritmos de enrutamiento. Su función principal será facilitar la entrega de los paquetes entre ambos extremos de la comunicación, de acuerdo con las características del protocolo que se defina.

La **capa de enlace de datos** tendrá como responsabilidad organizar los paquetes recibidos de la capa de red en tramas y controlar su transmisión. Para ello, se utilizará un protocolo de ventana deslizante que gestione las confirmaciones de recepción, el control de flujo y las retransmisiones cuando sea necesario. Adicionalmente, se implementará un algoritmo Checksum para detectar posibles alteraciones en la información recibida y determinar cuándo una trama no cumple con las condiciones de integridad establecidas.

La **capa física** será responsable de la transmisión de bits entre los dos Arduino mediante comunicación serial UART sobre el enlace de cobre STP. En esta capa se incorporará un mecanismo de codificación y decodificación basado en códigos de Hamming, con el objetivo de corregir determinados errores de bits producidos durante la transmisión.

3) Descomposición modular: Cada capa se dividirá en módulos especializados que permitan separar sus principales responsabilidades. Esta descomposición facilitará el desarrollo independiente de los componentes, así como la identificación de sus entradas, salidas e interfaces de comunicación.

En la capa de aplicación se contemplan módulos para la gestión de archivos, la configuración de los parámetros de comunicación y la supervisión de las transferencias. Estos componentes permitirán seleccionar los archivos que se desean transmitir, establecer las condiciones de comunicación y presentar al usuario la información correspondiente al estado del sistema.

En la capa de red se propone un módulo de gestión de paquetes, encargado de recibir la información proveniente de la capa de aplicación y organizar su entrega a la capa de enlace de datos. También proporcionará la interfaz necesaria para recibir los paquetes recuperados por la capa de enlace y entregarlos a la aplicación correspondiente. Se propone que este módulo realice la segmentación de datos sin un encabezado de direccionamiento, decisión que deberá validarse con el profesor.

En la capa de enlace de datos se consideran módulos para la construcción e interpretación de tramas, la detección de errores mediante Checksum y la gestión del protocolo de ventana deslizante. Este último deberá administrar las ventanas de transmisión, los números de secuencia, las confirmaciones de recepción y los mecanismos de retransmisión.

Finalmente, la capa física contará con módulos de codificación y decodificación Hamming, transmisión y recepción serial UART y configuración de los parámetros físicos del enlace. Estos módulos permitirán preparar la información para su transmisión, procesar los datos recibidos y gestionar la comunicación mediante los conductores TX y RX.

La descomposición propuesta se representará mediante diagramas de bloques de primer, segundo y tercer nivel. El primer nivel mostrará la arquitectura general del sistema y la conexión entre ambas terminales; el segundo identificará los principales módulos que conforman cada capa; y el tercero detallará el funcionamiento interno de los módulos, sus interfaces y los algoritmos que utilizarán. En todos los niveles se identificarán las capas del modelo OSI correspondientes.

4) Comunicación entre módulos: La interacción entre las capas se realizará mediante interfaces definidas que permitan intercambiar información sin exponer los detalles internos de cada componente. Cada capa utilizará los servicios proporcionados por la capa inmediatamente inferior y ofrecerá sus propios servicios a la capa superior, manteniendo una separación entre las diferentes responsabilidades del sistema.

Durante una transferencia, la capa de aplicación gestionará el archivo seleccionado y entregará los datos a la capa de red. Esta última se encargará de gestionar los paquetes de información y entregarlos a la capa de enlace de datos, que los organizará en tramas y gestionará su transmisión mediante el protocolo de ventana deslizante. Posteriormente, la computadora entregará los bytes de cada trama al Arduino mediante fragmentos controlados. El microcontrolador aplicará la codificación Hamming correspondiente y los transmitirá por el enlace de cobre STP.

En la terminal receptora se realizará el proceso complementario. El Arduino recibirá y decodificará la información mediante Hamming y la enviará progresivamente a la computadora; la capa de enlace comprobará la integridad de las tramas mediante Checksum y gestionará las confirmaciones o retransmisiones necesarias. Una vez recuperados los datos, estos se entregarán a la capa de red, que gestionará los paquetes recibidos y los proporcionará a la capa de aplicación para la reconstrucción del archivo original.

Dado que la comunicación es Full Duplex, ambas terminales podrán ejecutar los procesos de transmisión y recepción simultáneamente. Esto permitirá que cada dispositivo envíe

información mientras recibe tramas o confirmaciones procedentes de la otra terminal, de acuerdo con las reglas establecidas por el protocolo de comunicación.

B. Descripción del funcionamiento de cada módulo

El sistema de transferencia de archivos se organizará en módulos especializados, distribuidos entre las capas de aplicación, red, enlace de datos y física del modelo OSI. Cada módulo tendrá responsabilidades específicas e interfaces definidas para intercambiar información con los demás componentes del sistema. Esta distribución permitirá separar la gestión de archivos, el manejo de paquetes, el control de la transmisión y la comunicación mediante el medio físico. [1]

a) *Capa de aplicación:*

La capa de aplicación constituirá el punto de interacción entre el usuario y el sistema de transferencia de archivos. Su funcionamiento se organizará en tres módulos principales: gestión de archivos, configuración de comunicación y supervisión de transferencias.

1) Módulo de gestión de archivos:

Este módulo permitirá al usuario seleccionar los archivos que desea transmitir y administrará la recepción y reconstrucción de los archivos provenientes de la terminal remota. El sistema admitirá archivos de texto plano (TXT), documentos PDF e imágenes PNG, los cuales se procesarán como secuencias de bytes, independientemente de su formato.

Durante la transmisión, el módulo abrirá el archivo seleccionado, determinará su tamaño y entregará progresivamente su contenido a la capa de red. Durante la recepción, reconstruirá el archivo a partir de los datos recuperados por las capas inferiores, respetando el orden original de los bytes.

Para comprobar la integridad del archivo completo, se utilizará el algoritmo SHA-256. El emisor calculará una huella del archivo original y la transmitirá durante el procedimiento de finalización. El receptor calculará la misma huella sobre el archivo reconstruido y comparará ambos resultados. La transferencia se considerará exitosa únicamente cuando se haya recibido la cantidad esperada de información y ambas huellas coincidan.

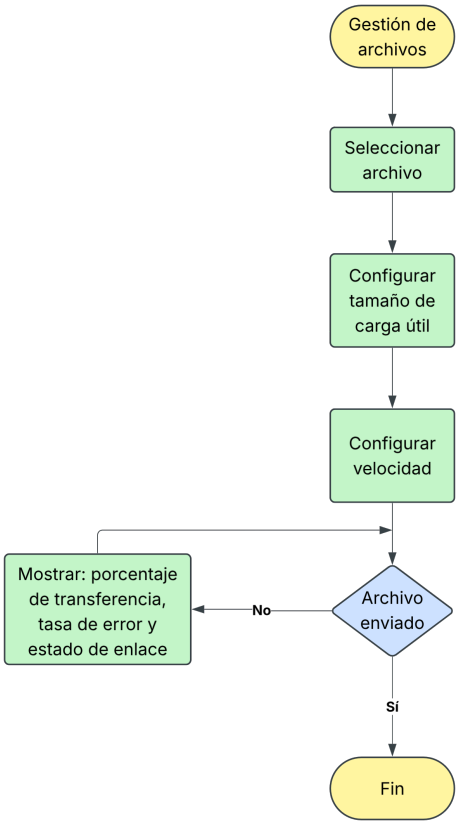


Fig. 1. Gestión de archivos

Submódulo de configuración de comunicación:

Este módulo permitirá establecer los parámetros necesarios para realizar cada transferencia. Se contemplarán las velocidades de transmisión de 9600 y 19200 bps, los tamaños de carga útil de 500 y 900 bytes y los anchos de ventana de 4, 6 y 8 tramas. Adicionalmente, permitirá activar o desactivar la codificación Hamming para realizar las pruebas comparativas correspondientes.

Antes de iniciar una transferencia, ambas terminales deberán configurar una velocidad de comunicación compatible. Posteriormente, intercambiarán mensajes de control para verificar que los demás parámetros seleccionados coincidan. Si se identifica alguna incompatibilidad, la solicitud será rechazada y se notificará al usuario.

Submódulo de supervisión de transferencias:

Este módulo proporcionará información sobre el estado de la comunicación y el progreso de las transferencias mediante una interfaz gráfica. Mostrará el estado del enlace, el porcentaje de transferencia completado y la tasa estimada de errores.

El monitor contemplará los siguientes mensajes de estado:

- Inicio de transmisión…
- Transmisión en curso…
- Estado de enlace óptimo…
- Estado de enlace ruidoso…
- Transmisión Finalizada!
- Tasa de error estimada: XX.xx %

El estado óptimo se mostrará cuando la tasa estimada de errores sea cero. El estado ruidoso se utilizará cuando se presenten errores irrecuperables durante la transmisión. La

tasa de error se presentará como un porcentaje con dos cifras decimales.

Para obtener estas métricas, el módulo utilizará los registros proporcionados por los mecanismos de detección de errores, decodificación Hamming y control de retransmisiones. Se distinguirán los errores corregidos, los errores no corregibles y las tramas que debieron retransmitirse.

b) *Capa de red:*

La capa de red proporcionará una interfaz para intercambiar paquetes entre la aplicación y la capa de enlace de datos. Aunque el sistema utiliza una conexión punto a punto entre dos terminales, se conservará esta capa dentro de la arquitectura modular, sin implementar algoritmos de enrutamiento.

1) Módulo de gestión de paquetes:

El módulo recibirá los datos provenientes de la capa de aplicación y organizará su entrega a la capa de enlace de datos. Para ello, dividirá el flujo de información en paquetes compatibles con el tamaño de carga útil seleccionado.

Se utilizarán paquetes de 500 o 900 bytes, excepto cuando el último paquete contenga una cantidad menor correspondiente a los datos restantes del archivo. Cada paquete se entregará a la capa de enlace para su encapsulamiento y transmisión.

Durante la recepción, el módulo recibirá los paquetes recuperados por la capa de enlace y los entregará a la aplicación, conservando el orden de la información original.

Debido a que únicamente existen dos terminales conectadas directamente, no se implementarán tablas de enrutamiento ni algoritmos de selección de rutas. En esta propuesta tampoco se añadirán encabezados de direccionamiento lógico. Las funciones definitivas de esta capa deberán ajustarse a los requisitos establecidos por el profesor.

c) *Capa de enlace de datos:*

La capa de enlace de datos será responsable de proporcionar una transferencia confiable de tramas entre las dos terminales. Para ello, se implementará el protocolo de ventana deslizante Go-Back-N (GBN), acompañado de mecanismos de detección de errores, confirmaciones acumulativas, temporizadores y retransmisiones. [1]

Las funciones principales de esta capa se ejecutarán en las computadoras, que dispondrán de la memoria necesaria para almacenar las ventanas de transmisión y los datos pendientes de confirmación.

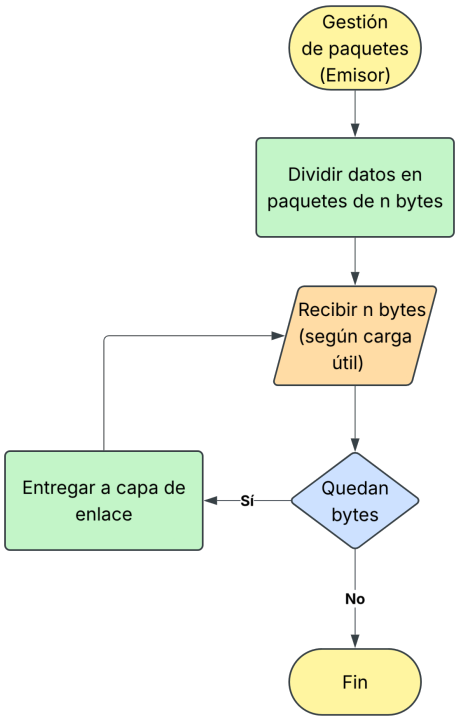


Fig. 2. Gestión de paquetes

1) Módulo de construcción e interpretación de tramas:

Este módulo recibirá los paquetes provenientes de la capa de red y los encapsulará en tramas para su transmisión. Se utilizará un formato común para las tramas de datos y de control, compuesto por los siguientes campos:

- Delimitador de inicio: 1 byte.
- Campo de control: 1 byte.
- Número de secuencia: 1 byte.
- Número de confirmación ACK: 1 byte.
- Longitud: 2 bytes.
- Carga útil: de 0 a 900 bytes.
- Checksum: 2 bytes.
- Delimitador de fin: 1 byte.

Los delimitadores de inicio y fin utilizarán el valor hexadecimal 0x7E. El campo de control identificará el tipo de trama transmitida, mientras que los campos de secuencia y ACK permitirán gestionar la numeración y las confirmaciones del protocolo Go-Back-N.

Los campos de secuencia y confirmación utilizarán sus cuatro bits menos significativos para representar números comprendidos entre 0 y 15. El campo de longitud tendrá dos bytes, transmitidos en orden de mayor a menor significancia, y especificará la cantidad de bytes presentes en la carga útil.

Durante la recepción, el módulo interpretará los campos de cada trama, comprobará su estructura y verificará que su longitud sea válida. Las tramas que superen las comprobaciones de integridad serán procesadas según su tipo y, cuando corresponda, entregadas a la capa de red.

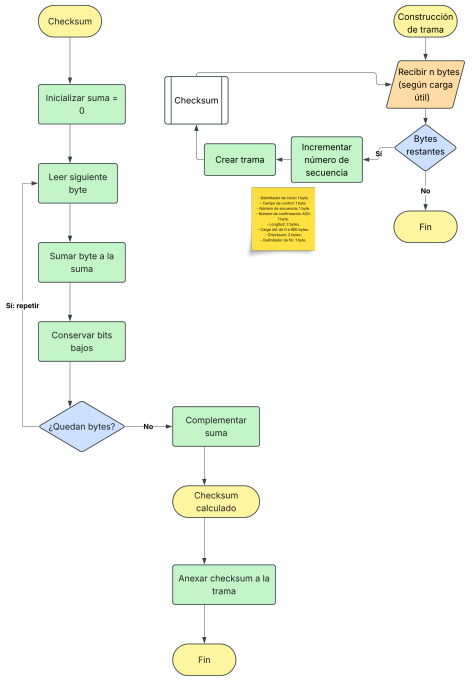


Fig. 3. Construcción de trama

2) Tipos de trama:

El campo de control tendrá una longitud de un byte y permitirá identificar los diferentes mensajes del protocolo. Se utilizarán los siguientes valores (Ver Fig. 10):

- 0x01: DATA, para transportar paquetes de información.
- 0x02: ACK, para enviar confirmaciones acumulativas.
- 0x03: START, para solicitar el inicio de una transferencia.
- 0x04: FIN, para solicitar la finalización de una transferencia.
- 0x05: ABORT, para cancelar una transferencia.
- 0x06: START_ACK, para confirmar o rechazar una solicitud de inicio.
- 0x07: FIN_ACK, para comunicar el resultado de la finalización.

Durante la implementación inicial, las confirmaciones se enviarán mediante tramas ACK independientes, sin utilizar piggybacking. En las tramas DATA, el campo ACK tendrá el valor reservado 0xFF, que indicará que no contiene una confirmación válida.

Las tramas de control distintas de DATA utilizarán el valor reservado 0xFF en el campo de secuencia y no consumirán posiciones de la ventana Go-Back-N. Los mensajes de establecimiento y finalización contarán con temporizadores independientes y mecanismos para reconocer mensajes repetidos.

Submódulo de detección de errores:

Este módulo utilizará un Checksum de 16 bits, calculado mediante la suma de palabras de 16 bits con complemento a uno. El cálculo incluirá los campos de control, secuencia, ACK, longitud y carga útil.

Si la cantidad de bytes protegidos es impar, se añadirá un byte de relleno con valor cero únicamente para realizar el cálculo. El resultado se complementará a uno y se incorporará al campo Checksum de la trama.

Durante la recepción, se realizará la comprobación correspondiente mediante el mismo algoritmo. Si la verificación falla, la trama será descartada y no se entregará a las capas superiores.

El Checksum se complementará con los códigos de Hamming implementados en la capa física. Mientras Hamming permitirá corregir determinados errores de bits, el Checksum permitirá detectar alteraciones que permanezcan en la trama después de la decodificación.

4) Módulo de gestión de ventana deslizante:

Se utilizará el protocolo Go-Back-N, que permite transmitir varias tramas consecutivas sin esperar una confirmación individual después de cada envío. El emisor mantendrá una ventana configurable de 4, 6 u 8 tramas, dependiendo de los parámetros establecidos para cada prueba.

Cada trama de datos tendrá un número de secuencia de cuatro bits, con valores comprendidos entre 0 y 15. La numeración será circular, por lo que después de la trama 15 continuará nuevamente la trama 0.

El emisor mantendrá dos variables principales: Base, que identificará la trama más antigua pendiente de confirmación, y NextSeqNum, que representará el siguiente número de secuencia disponible.

Mientras exista espacio dentro de la ventana, el emisor podrá transmitir nuevas tramas. Cada trama pendiente deberá conservarse en el búfer de retransmisión hasta recibir una confirmación acumulativa que permita liberarla.

El receptor mantendrá una variable ExpectedSeqNum, correspondiente al siguiente número de secuencia esperado. Únicamente aceptará las tramas válidas que coincidan con dicho número; las tramas recibidas fuera de orden serán descartadas.

Cuando el receptor acepte una trama, actualizará ExpectedSeqNum y enviará una confirmación acumulativa. Si recibe una trama duplicada o fuera de secuencia, repetirá la confirmación correspondiente al siguiente número esperado.

Al recibir una confirmación acumulativa válida, el emisor desplazará su ventana y podrá transmitir nuevas tramas.

Cada dirección de comunicación dispondrá de una instancia independiente de Go-Back-N, permitiendo que las dos terminales actúen simultáneamente como emisoras y receptoras.

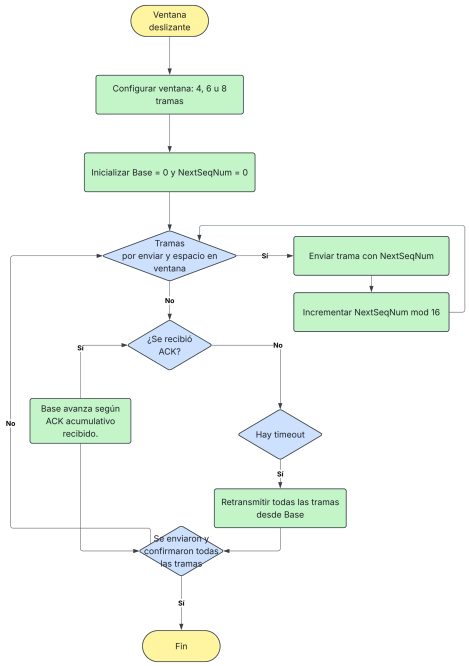


Fig. 4. Ventana deslizante

Submódulo de confirmaciones, temporizadores y retransmisiones:

El protocolo utilizará confirmaciones acumulativas. El campo ACK identificará el siguiente número de secuencia que el receptor espera recibir, reconociendo de manera acumulativa las tramas anteriores que se encuentren pendientes dentro de la ventana.

Las tramas ACK independientes incluirán el identificador de transferencia de cuatro bytes en su carga útil, para evitar que una confirmación de otra operación sea interpretada incorrectamente. El campo de secuencia de estas tramas será 0xFF y el campo ACK contendrá el siguiente número esperado.

El emisor mantendrá un temporizador asociado a la trama más antigua que permanezca sin confirmar. El tiempo de espera comenzará cuando el Arduino emisor informe que terminó de transmitir físicamente dicha trama, no cuando la computadora finalice la entrega de los fragmentos por USB.

Si el temporizador expira, el emisor retransmitirá todas las tramas pendientes, comenzando desde la más antigua que no haya sido confirmada. Cuando reciba una confirmación que desplace la ventana, actualizará Base y reiniciará o detendrá el temporizador según corresponda.

La duración del temporizador deberá considerar la velocidad de transmisión, el tamaño de las tramas, la sobrecarga introducida por Hamming y el tiempo necesario para procesar y confirmar la información. Su valor inicial se calculará antes de la implementación y se ajustará mediante pruebas experimentales.

Adicionalmente, se establecerá un límite configurable de reintentos. Si una trama continúa sin confirmarse después de superar ese límite, la transferencia se considerará fallida y se notificará el error a la aplicación.

6) Módulo de establecimiento de comunicación:

Antes de comenzar el intercambio de datos, el sistema realizará un procedimiento de establecimiento mediante los mensajes START y START_ACK. El objetivo será identificar la transferencia, verificar los parámetros de comunicación y confirmar que ambas terminales estén preparadas.

La carga útil del mensaje START tendrá una longitud de 14 bytes, distribuidos de la siguiente manera:

- Identificador de transferencia: 4 bytes.
- Tamaño del archivo: 4 bytes.
- Tipo de archivo: 1 byte.
- Código de velocidad: 1 byte.
- Tamaño de carga útil: 2 bytes.
- Ancho de ventana: 1 byte.
- Modo Hamming: 1 byte.

El identificador permitirá distinguir diferentes transferencias y reconocer solicitudes repetidas. El tamaño del archivo indicará la cantidad total de bytes que se desea transmitir.

El tipo de archivo se representará mediante los valores 0x01 para TXT, 0x02 para PDF y 0x03 para PNG. La velocidad utilizará 0x01 para 9600 bps y 0x02 para 19200 bps. El modo Hamming utilizará 0x00 cuando esté desactivado y 0x01 cuando esté activado.

La velocidad UART y el modo Hamming deberán configurarse previamente en ambas terminales mediante un procedimiento común. START comprobará que los parámetros coincidan, pero no realizará cambios automáticos de velocidad ni de codificación.

Al recibir una solicitud START, la terminal receptora verificará los parámetros y responderá mediante START_ACK. Su carga útil contendrá el identificador de transferencia de cuatro bytes y un código de estado de un byte, en el que 0x00 representará aceptación y los demás valores indicarán condiciones de rechazo.

Si el emisor no recibe START_ACK antes de que expire el temporizador correspondiente, retransmitirá la solicitud. Cuando el receptor identifique un START repetido perteneciente a una transferencia ya establecida, reenviará la confirmación sin crear una operación duplicada.

La transmisión de datos comenzará únicamente después de recibir la confirmación de establecimiento.

7) Módulo de finalización de transferencia:

Una vez confirmadas todas las tramas de datos, el emisor iniciará el procedimiento de finalización mediante una trama FIN. Este mensaje tendrá una carga útil de 36 bytes, compuesta por un identificador de transferencia de cuatro bytes y una huella SHA-256 de 32 bytes correspondiente al archivo original.

El receptor comprobará que haya recibido la cantidad de información indicada en START. Posteriormente, calculará SHA-256 sobre el archivo reconstruido y comparará el resultado con la huella recibida mediante FIN.

Si la comprobación es satisfactoria, el receptor enviará FIN_ACK, incluyendo el identificador de transferencia y un código que indique que la operación finalizó correctamente. Si la comprobación falla, la transferencia se considerará fallida y se notificará el error.

El mensaje FIN utilizará un temporizador independiente. Si se pierde FIN_ACK, el emisor podrá retransmitir FIN. El

receptor conservará temporalmente el resultado de la transferencia para responder a solicitudes repetidas sin volver a procesar el archivo.

Adicionalmente, se utilizará el mensaje ABORT para cancelar una transferencia cuando se produzca una condición de error irrecuperable o el usuario solicite interrumpir la operación. Su carga útil contendrá el identificador de transferencia y un código que indique la causa de cancelación.

8) Módulo de administración de memoria:

La gestión de las ventanas de transmisión y los búferes de retransmisión se realizará en las computadoras, debido a las restricciones de memoria de los microcontroladores.

Una ventana de ocho tramas con cargas útiles de 900 bytes requiere almacenar al menos 7200 bytes de información, sin considerar encabezados ni estructuras de control. Esta cantidad supera la memoria SRAM disponible en los modelos clásicos de Arduino Uno y Nano.

Por tanto, las computadoras conservarán las tramas pendientes de confirmación, mientras que los Arduino procesarán los datos de manera incremental mediante fragmentos pequeños.

Esta distribución permitirá utilizar los diferentes anchos de ventana sin que los microcontroladores necesiten almacenar tramas completas en su memoria.

d) *Capa física:*

La capa física será responsable de transmitir y recibir los bits mediante el enlace de cobre STP que interconecta ambos Arduino. Se utilizará comunicación serial UART a velocidades de 9600 y 19200 bps y codificación Hamming extendida (13,8) para proporcionar corrección de errores. [2]

La arquitectura propuesta contempla un controlador UART adicional conectado mediante SPI, que permitirá mantener una comunicación independiente con la computadora y establecer un enlace Full Duplex entre los microcontroladores.

1) Módulo de codificación y decodificación Hamming:

Se utilizará Hamming extendido (13,8), también denominado SECDED, con capacidad de corregir errores de un bit y detectar errores de dos bits por bloque de ocho bits de información.

Cada bloque codificado contendrá ocho bits de datos, cuatro bits de paridad Hamming y un bit adicional de paridad global. Los bits de paridad Hamming ocuparán las posiciones 1, 2, 4 y 8, mientras que los bits de información se distribuirán en las posiciones 3, 5, 6, 7, 9, 10, 11 y 12. El bit de paridad global ocupará la posición 13.

Se utilizará paridad par para realizar las comprobaciones. Cada bloque de trece bits se representará mediante dos bytes durante su transporte por UART, estableciendo en cero los tres bits que no se utilicen.

Durante la recepción, se calculará el síndrome Hamming y se verificará la paridad global. A partir de estos resultados, se distinguirán cuatro situaciones:

- Síndrome igual a cero y paridad global correcta: no se detectaron errores.
- Síndrome distinto de cero y paridad global incorrecta: error corregible de un bit.
- Síndrome igual a cero y paridad global incorrecta: error en el bit de paridad global.
- Síndrome distinto de cero y paridad global correcta: error doble detectado.

Cuando se identifique un error corregible, se recuperará el dato original. Si se detecta un error doble, el bloque se marcará como no recuperable y la trama correspondiente se invalidará. La capacidad SECDED garantizada se limita a errores de uno y dos bits; los patrones con más errores pueden no detectarse correctamente.

También se implementará un modo de transmisión sin Hamming, que permitirá efectuar las pruebas comparativas exigidas por el proyecto.

2) Módulo de transmisión y recepción UART:

Cada Arduino utilizará un controlador UART externo conectado mediante SPI para establecer la comunicación con el otro microcontrolador. Se propone utilizar un SC16IS750 o un componente de características equivalentes.

La conexión USB de cada Arduino se reservará para el intercambio de información con su computadora, mientras que el controlador UART adicional se utilizará para transmitir y recibir información mediante el enlace de cobre.

La transmisión y la recepción funcionarán de manera independiente para proporcionar comunicación Full Duplex. El controlador UART se configurará con las velocidades de 9600 o 19200 bps, de acuerdo con los parámetros seleccionados para cada prueba.

Durante la implementación deberán verificarse la compatibilidad eléctrica del controlador UART con los microcontroladores, la posible adaptación de niveles lógicos y la disponibilidad real del módulo propuesto. La conexión entre los TX y RX de los controladores externos constituirá el enlace STP; los pines SPI solo conectarán cada controlador con su Arduino.

La incorporación del controlador adicional estará sujeta a la aprobación del profesor.

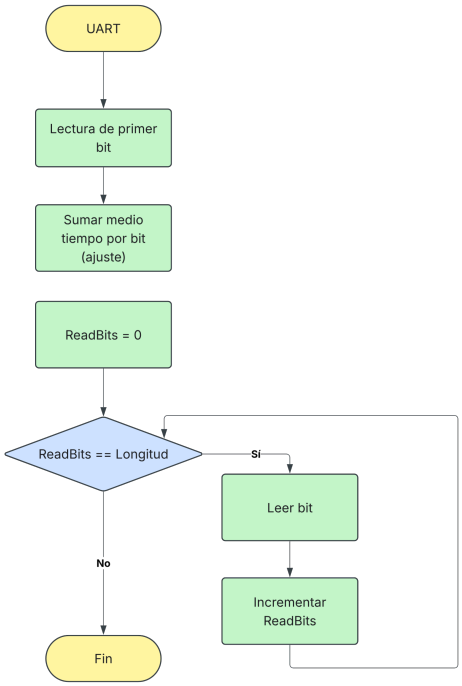


Fig. 5. UART

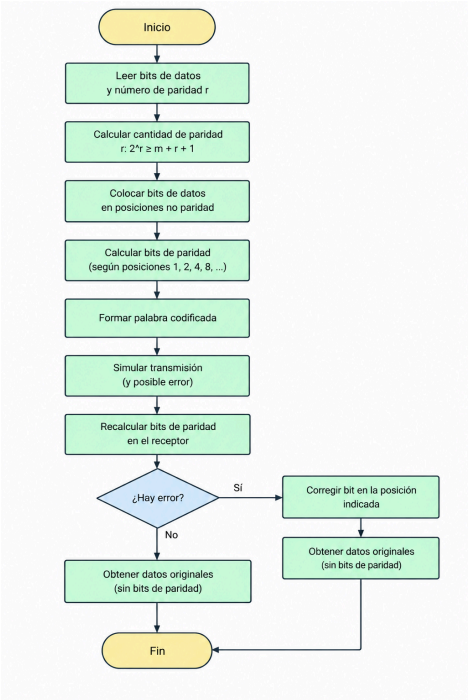


Fig. 6. Código hamming

3) Módulo de delimitación y escape:

Se utilizará el valor hexadecimal 0x7E como delimitador de inicio y fin de las tramas. Los delimitadores se transmitirán sin codificación Hamming, permitiendo que el receptor identifique los límites de cada trama.

En la terminal emisora, la computadora construirá primero la trama lógica y calculará su Checksum. Posteriormente, entregará los bytes de la trama al Arduino, que aplicará la codificación Hamming cuando este mecanismo se encuentre activado.

Los bloques codificados se representarán mediante dos bytes cada uno. Antes de transmitirlos, se aplicará un mecanismo de escape para evitar que los valores internos puedan confundirse con los delimitadores.

Cuando aparezca el valor 0x7E o 0x7D, se transmitirá primero el byte de escape 0x7D, seguido del byte original modificado mediante una operación XOR con 0x20.

En el receptor se realizará el proceso inverso: se identificarán los delimitadores, se eliminarán los escapes y se decodificarán los bloques Hamming. Los bytes recuperados se entregarán progresivamente a la computadora, donde la capa de enlace verificará el Checksum.

Si se detecta un error no corregible durante la decodificación, la trama se marcará como inválida. El receptor continuará procesando el flujo físico hasta encontrar el delimitador de fin, pero no entregará la trama dañada a las capas superiores.

En las pruebas sin Hamming se mantendrá el mismo mecanismo de delimitación y escape, sin aplicar la codificación correctora.

4) Módulo de comunicación local y fragmentación:

La comunicación entre cada computadora y su Arduino se realizará mediante USB serial. Para evitar el desbordamiento de la memoria del microcontrolador, los datos se dividirán en fragmentos de hasta 32 bytes de contenido.

Se utilizará un protocolo local con mensajes para iniciar una trama, transferir sus fragmentos, indicar el final de la trama y confirmar la disponibilidad del receptor.

Cada fragmento incluirá un identificador de secuencia local, un campo de longitud y una comprobación de integridad local, cuya representación binaria deberá precisarse antes de la implementación. Este protocolo será independiente del utilizado por Go-Back-N entre las dos terminales.

Durante la transmisión, la computadora enviará un fragmento y esperará a que el Arduino confirme que puede aceptar el siguiente. El microcontrolador procesará los bytes recibidos y los transmitirá progresivamente mediante el controlador UART externo.

El Arduino deberá preservar el orden de los fragmentos y no mezclar los bytes pertenecientes a tramas diferentes. Una trama física deberá completarse antes de comenzar la transmisión de la siguiente.

Durante la recepción, el Arduino realizará el procedimiento inverso, recuperando los datos provenientes del enlace y entregándolos a su computadora mediante fragmentos controlados.

Las confirmaciones del protocolo local no equivaldrán a las confirmaciones de Go-Back-N. Una confirmación local indicará únicamente que el fragmento se procesó o quedó almacenado de forma segura para continuar su transmisión.

Por su parte, un ACK de Go-Back-N únicamente se generará cuando la computadora receptora haya recibido y verificado correctamente la trama completa.

5) Módulo de configuración y supervisión del enlace:

Este módulo administrará los parámetros del controlador UART y supervisará los eventos producidos durante la comunicación física.

Se registrarán los bloques Hamming con errores corregidos, los errores no corregibles, las tramas rechazadas por Checksum y las retransmisiones realizadas por Go-Back-N.

Estas estadísticas se entregarán a la aplicación para calcular los indicadores de transmisión y representar el estado del enlace.

La tasa estimada de tramas con errores detectados se calculará a partir de la cantidad de tramas dañadas y del número total de tramas recibidas. Esta métrica no debe confundirse con la tasa de errores de bits. Las correcciones realizadas por Hamming se contabilizarán de forma independiente.

Cuando se disponga de los datos originales para efectuar una comparación, también se calculará la tasa de errores de bits.

6) Módulo de interconexión física:

El enlace de cobre estará formado por tres conductores: TX, RX y GND. El conductor TX del primer dispositivo se conectará al RX del segundo, mientras que el TX del segundo se conectará al RX del primero. Ambos dispositivos compartirán una referencia de tierra común.

Se utilizará cable de cobre blindado STP, de acuerdo con la configuración asignada al grupo. El enlace tendrá una longitud de entre ocho y diez metros, siguiendo las condiciones de construcción establecidas en la especificación del proyecto.

El par TX-RX deberá trenzarse según las indicaciones del profesor y contar con una unión intermedia mediante trenzado, sin soldadura, para simular condiciones adversas del medio físico.

Durante la implementación se verificarán la continuidad de los conductores, la conexión de tierra, el tratamiento eléctrico del blindaje y el funcionamiento del enlace a ambas velocidades de transmisión.

La construcción del enlace permitirá evaluar el comportamiento de los mecanismos de detección y corrección de errores ante las condiciones físicas previstas para el proyecto.

III. Resumen de resultados esperados

A partir del diseño propuesto, se espera obtener un sistema capaz de transferir archivos TXT, PDF y PNG entre las dos terminales de forma confiable, aprovechando la capacidad Full Duplex del enlace.

Se espera que el sistema complete transferencias de los tres tipos de archivo con todas las combinaciones de parámetros definidas. Una transferencia se considerará correcta cuando el receptor obtenga la cantidad de bytes indicada en START y la huella SHA-256 calculada sobre el archivo reconstruido coincida con la recibida en FIN.

1) Influencia de la velocidad UART en la calidad del enlace

Se espera que las pruebas realizadas a 19200 bps presenten un enlace menos estable que las realizadas a 9600 bps. Al duplicar la velocidad de transmisión, cada bit dura la mitad de tiempo, por lo que cualquier distorsión o alteración en la señal puede afectar una mayor proporción del bit y dificultar su correcta interpretación por parte del receptor UART. Aunque el cable STP ayuda a reducir las interferencias externas [1], la unión intermedia trenzada y sin soldadura continúa siendo un punto vulnerable, ya que puede presentar pequeñas variaciones en la resistencia o en el contacto. Por esta razón, a 19200 bps se espera encontrar una mayor cantidad de tramas descartadas y más vencimientos del temporizador de Go-Back-N, lo que podría aumentar el tiempo total de transferencia más de lo esperado considerando únicamente el aumento de velocidad.

2) Menor cantidad de retransmisiones con Hamming extendido (13,8)

El modo SECDED corrige en la propia Capa Física los bloques afectados por un único bit erróneo y marca como inválidos aquellos con dos bits alterados. Por lo que se espera que una parte importante de los errores producidos por el empalme quedará resuelta en el Arduino receptor, y la computadora recibirá menos tramas con un Checksum incorrecto. Este efecto se espera ver especialmente en Go-Back-N: como el diseño no contempla confirmaciones negativas, cada trama perdida obliga al emisor a esperar el vencimiento del temporizador y reenviar todas las tramas pendientes a partir de ella. Sin Hamming, cualquier bit alterado provocará el descarte de la trama completa, por lo que las estadísticas de retransmisión deberían ser más altas en ese modo.

3) Errores que pueden escapar a la verificación de la trama

El Checksum de 16 bits basado en complemento a uno es sencillo de calcular, pero no garantiza detectar todas las alteraciones. Para cubrir estos casos, el diseño incorpora la huella SHA-256 del archivo completo, verificada durante la finalización, de modo que una transferencia con datos alterados se identifique como fallida en lugar de reportarse como exitosa. [2]

IV. Recomendaciones para la siguiente etapa

Con la arquitectura del sistema organizada mediante diseño modular, y con los protocolos de las capas física, de enlace de datos, de red y de aplicación ya especificados, se proponen las siguientes consideraciones para orientar la implementación y validación de la solución en la Segunda Etapa:

- Se recomienda trabajar por etapas, de manera de que se pueda probar primero cada módulo por separado, comenzando por Hamming, la delimitación y escape y la comunicación UART. Después se pueden integrar las tramas, el Checksum y finalmente Go-Back-N. Esto facilita encontrar los errores sin tener que depurar todo el sistema al mismo tiempo.
- Se debe de revisar el empalme del cable STP, la unión trenzada sin soldadura puede convertirse en un punto débil. Es importante mantener correctamente el blindaje y su conexión a tierra para que el cable conserve la protección contra interferencias.
- Se recomienda mantener las pruebas controladas. Al comparar Hamming, la ventana o la velocidad, se deben mantener constantes los demás parámetros y modificar únicamente el que se quiere evaluar. Así, las diferencias

en los resultados podrán relacionarse directamente con ese cambio.

- Los modelos Uno y Nano deben compartirse los datos del protocolo, la biblioteca del SC16IS750 y las variables del programa. Por lo que se recomienda controlar la memoria disponible y evitar la asignación dinámica para prevenir errores difíciles de identificar.

V. Anexos

Los diagramas se encuentran en: https://lucid.app/lucidchart/2b76edca-2e71-44bc-8edb-ffd60d3e604c/view?page=hFwzAh7B3HoR#

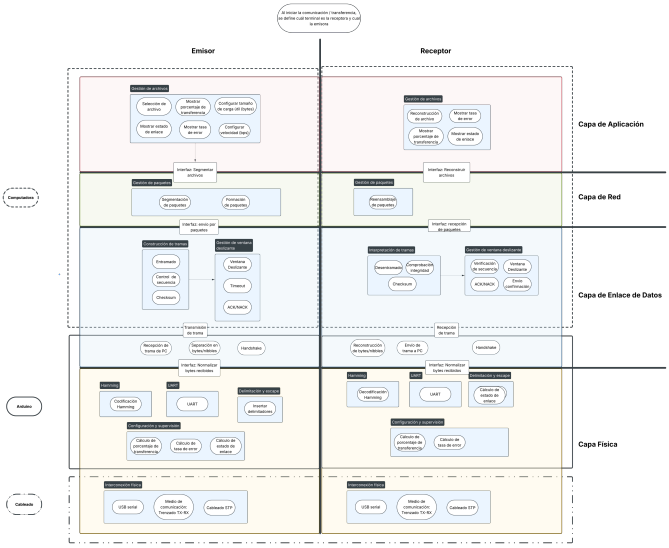


Fig. 7. Diagrama Bloques Nivel 3

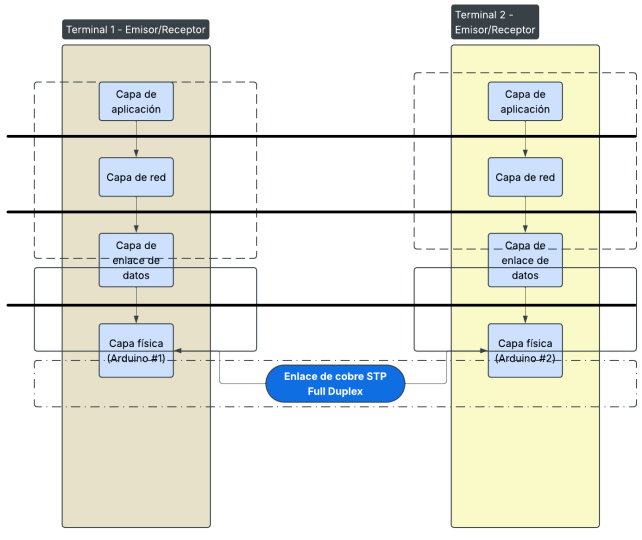


Fig. 8. Diagrama Bloques Nivel 1

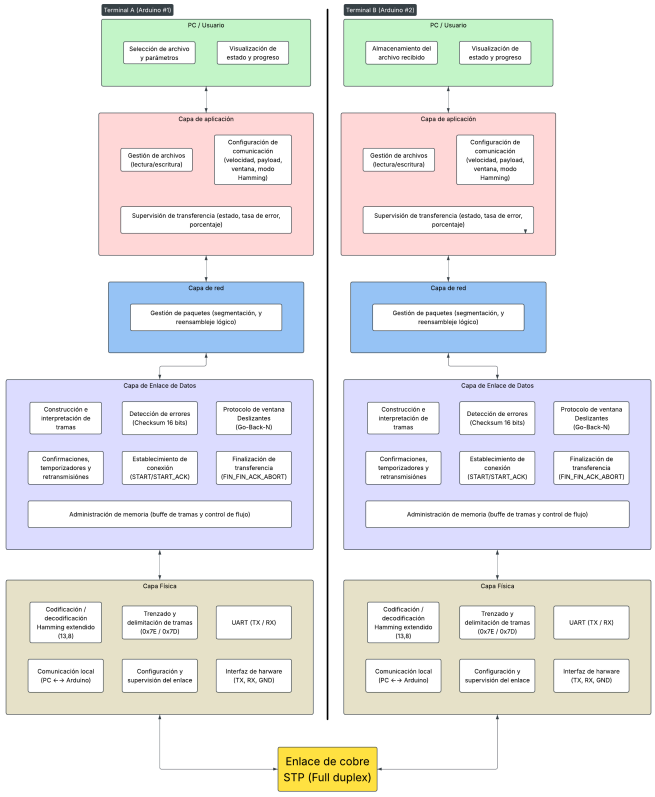


Fig. 9. Diagrama Bloques Nivel 2

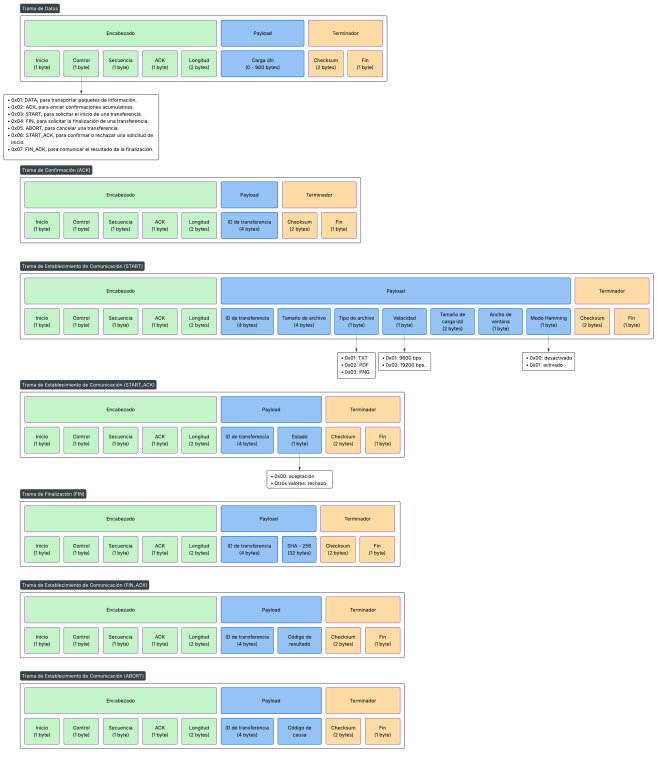


Fig. 10. Tipos de Tramas

References

[1] A. S. Tanenbaum y D. J. Wetherall, *Redes de computadoras*, 5.ª ed. Pearson Educación, 2012.

[2] R. W. Hamming, «Error Detecting and Error Correcting Codes», *The Bell System Technical Journal*, vol. 29, n.º 2, pp. 147-160, 1950.